

Vers React : limites des Web Components et gestion de l'état

Objectifs de cette section

Les Web Components offrent une vraie modularité, mais ils atteignent vite leurs limites quand l'interface devient complexe et que **l'état de l'application** doit être synchronisé entre plusieurs composants.

Dans cette section, vous allez :

- Comprendre les limites des Web Components "nus".
- Identifier les problèmes liés à la gestion de l'état.
- Mettre en place une communication minimale entre composants.
- Préparer la transition vers une bibliothèque comme **React**.

1. Les limites des Web Components

Les Web Components sont puissants, mais ils n'offrent pas nativement :

- Une gestion facile de **l'état partagé** entre composants.
- Une mise à jour automatique de l'interface quand les données changent.
- Une approche déclarative du rendu (comme avec JSX ou Vue/React).

Dès que l'interface devient **interactive à plusieurs niveaux**, la gestion du DOM et des événements devient lourde.

2. Comprendre la notion d'état

Un **état** est l'ensemble des données qui définissent ce que doit afficher un composant à un instant donné.

Exemples d'état :

- Le texte d'un champ de formulaire.
- La liste des tâches.
- Une valeur booléenne "complété / non complété".

Exemple : afficher un état booléen dans un composant

```
class StateDisplay extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
    this.state = {active: false};
    this.render();
  }

  toggleState() {
    this.state.active = !this.state.active;
    this.render();
  }

  render() {
    this.shadowRoot.innerHTML = `
    <p>État : ${this.state.active ? "Actif" : "Inactif"}</p>
    <button>Changer d'état</button>
    `;

    this.shadowRoot.querySelector("button").addEventListener("click", () => this.toggleState());
  }
}

customElements.define("state-display", StateDisplay);
```

3. Modifier l'état et mettre à jour l'affichage

Dans une architecture manuelle, on doit :

- Mettre à jour la donnée (ex : une variable).
- Puis **manuellement** mettre à jour le DOM pour refléter cette modification.

Exemple : basculer une tâche comme complétée / non complétée

```
class ToggleTask extends HTMLElement {
  constructor() {
    super();
    this.attachShadow({mode: "open"});
    this.state = {done: false};
    this.render();
  }

  toggle() {
    this.state.done = !this.state.done;
    this.render();
  }

  render() {
    const textStyle = this.state.done ? "text-decoration: line-through;" : "";
    this.shadowRoot.innerHTML = `
<label style="${textStyle}">
  <input type="checkbox" ${this.state.done ? "checked" : ""}>
    ${this.getAttribute("text") || "Tâche"}
</label>
`;

    this.shadowRoot.querySelector("input").addEventListener("change", () => this.toggle());
  }
}

customElements.define("toggle-task", ToggleTask);
```

4. Communiquer entre composants

Les Web Components ne partagent pas d'état automatiquement.

- Pour transmettre une information : on peut utiliser les **attributs**, des **custom events**, ou un **objet global** (à éviter).
- La synchronisation devient vite pénible à maintenir à mesure que l'application grossit.

Cette difficulté est une des raisons majeures de l'existence de bibliothèques comme **React**, qui gèrent cela de manière **automatique, déclarative et optimisée**.

5. Exercice de mise en pratique

Consignes

1. Reprenez le composant `<task-item>` que vous avez déjà créé.
2. Ajoutez une case à cocher (`<input type="checkbox">`) pour marquer la tâche comme complétée.
3. Lorsque la case est cochée ou décochée :
 - L'état interne du composant doit être mis à jour.
 - Le style de la tâche doit changer (ex. : texte barré si complété).

Bonus : faites que le style initial reflète correctement l'état (coché ou non) en lecture seule.

Organisation attendue

```
/exercices
├── 09-composant-etat
│   ├── index.html
│   └── toggle-task.js
```